

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
Departamento de Ciências de Computação
Disciplina de Algoritmos e Estruturas de Dados II (SCC0503)

Docente

Profa. Dra. Cristina Dutra de Aguiar
cdac@icmc.usp.br

Monitores

Gustavo Ramos Santos Pires

gustavo.rspires@usp.br ou telegram: @darrocaxd

João Gabriel Pieroli da Silva

joaogabrielpieroli@usp.br ou telegram: @bielcomp24

Pedro Lunkes Villela

pedrolunkesvillela@usp.br ou telegram: @pedrolunkes

Renan Banci Catarin

renanbcatarin@usp.br ou telegram: @Reckat

Renan Trofino Silva

renan.trofino@usp.br ou telegram: @renan823

Vinícius Souza Freitas

vininicius@usp.br ou telegram: @Vininicius

Trabalho Prático 1

Este trabalho tem como objetivo inserir, remover e atualizar dados de um arquivo binário.

O trabalho deve ser feito por, no máximo, 2 alunos da mesma turma. Os alunos devem ser os mesmos do trabalho introdutório. Quaisquer mudanças devem ser devidamente informadas. A solução deve ser proposta exclusivamente pelo(s) aluno(s) com base nos conhecimentos adquiridos nas aulas. Consulte as notas de aula e o livro texto quando necessário.

Descrição do Arquivo de Índice Primário

O arquivo de dados **estacao** é indexado por um índice primário, que é definido sobre o campo **codEstacao**. O índice primário também é um arquivo, sendo chamado de arquivo de índice **indexaEstacao**. A especificação de **indexaEstacao** é feita a seguir.

Registro de Cabeçalho. O registro de cabeçalho deve conter o seguinte campo:

- **status**: indica a consistência do arquivo de dados, devido à queda de energia, travamento do programa, etc. Pode assumir os valores '0', para indicar que o

arquivo de dados está inconsistente, ou '1', para indicar que o arquivo de dados está consistente. Ao se abrir um arquivo para escrita, seu *status* deve ser '0' e, ao finalizar o uso desse arquivo, seu *status* deve ser '1' – tamanho: *string* de 1 byte.

Representação Gráfica do Registro de Cabeçalho. O tamanho do registro de cabeçalho deve ser de 1 byte, representado da seguinte forma:



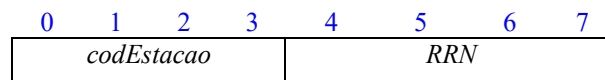
Observações Importantes.

- O registro de cabeçalho deve seguir estritamente a ordem definida na sua representação gráfica.
- Neste projeto, o conceito de página de disco não está sendo considerado.

Registros de Dados. Os registros de dados são de tamanho fixo, com campos de tamanho fixo, da seguinte forma:

- *codEstacao*: código sequencial que identifica univocamente cada registro de estação armazenado no arquivo de dados estacao: *inteiro* – *tamanho*: 4 bytes.
- *RRN*: *RRN* do registro do arquivo de dados que se refere ao *codEstacao* – *inteiro* – *tamanho*: 4 bytes.

Representação Gráfica dos Registros de Dados. O tamanho de cada registro de dados deve ser de 8 bytes, representado da seguinte forma:



Observações Importantes.

- Cada registro de dados deve seguir estritamente a ordem definida na sua representação gráfica.
- Os campos *codEstacao* e *RRN* não aceitam valores nulos.
- Os valores de *codEstacao* devem ser ordenados de forma crescente.
- Neste projeto, o conceito de página de disco não está sendo considerado.

Programa

Descrição Geral. Implemente um programa em C por meio do qual o usuário possa inserir, remover e atualizar dados de arquivos binários.

Importante. A definição da sintaxe de cada comando bem como sua saída devem seguir estritamente as especificações definidas em cada funcionalidade. Para especificar a sintaxe de execução, considere que o programa seja chamado de “programaTrab”. Essas orientações devem ser seguidas uma vez que a correção do funcionamento do programa se dará de forma automática. De forma geral, a primeira entrada da entrada padrão é sempre o identificador de suas funcionalidades, conforme especificado a seguir.

Modularização. É importante modularizar o código. Trechos de programa que aparecerem várias vezes devem ser modularizados em funções e procedimentos.

Descrição Específica. O programa deve oferecer as seguintes funcionalidades:

Na linguagem SQL, o comando CREATE INDEX é usado para criar um índice sobre um campo (ou um conjunto de campos) de busca. A funcionalidade [5] representa um exemplo de implementação da criação de um índice primário definido sobre o campo chave primária de um arquivo de dados.

[5] Crie um arquivo de índice primário para um arquivo de dados de entrada. O arquivo de índice primário deve ser gerado de acordo com as especificações da **Descrição do Arquivo de Índice Primário** definidas neste trabalho prático, e deve indexar corretamente o arquivo de dados criado na funcionalidade [1]. Isso significa que, no momento da criação, o arquivo de índice primário deve possuir um registro de cabeçalho e vários registros de dados.

Entrada do programa para a funcionalidade [5]:

```
5 arquivoEntrada.bin arquivoIndicePrimario.bin
```

onde:

- arquivoEntrada.bin é o arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório.
- arquivoIndicePrimario.bin é um arquivo de índice primário no formato binário, o qual é gerado de acordo com as especificações da **Descrição do Arquivo de Índice Primário** definidas neste trabalho prático.

Saída caso o programa seja executado com sucesso:

Listar todos os registros do arquivo de índice primário no formato binário usando a função fornecida binarioNaTela.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab  
5 estacao.bin indexaEstacao.bin
```

usar a função binarioNaTela antes de terminar a execução da funcionalidade, para mostrar a saída do arquivo indexaEstacao.bin.

Conforme visto nas funcionalidades [2] e [3], na linguagem SQL o comando SELECT é usado para listar os dados de uma tabela. Existem várias cláusulas que compõem o comando SELECT. Além das cláusulas SELECT e FROM, outra cláusula muito comum é a cláusula WHERE, que permite que seja definido um critério de busca sobre um ou mais campos, o qual é nomeado como critério de seleção.

SELECT lista de colunas (ou seja, campos a serem exibidos na resposta)

FROM tabela (ou seja, arquivo que contém os campos)

WHERE critério de seleção (ou seja, critério de busca)

A funcionalidade [6] representa um exemplo de implementação do comando SELECT considerando a cláusula WHERE com o apoio de um índice primário.

[6] Permita a recuperação dos dados de todos os registros de um arquivo de dados de entrada, de forma que esses registros satisfaçam um critério de busca determinado pelo usuário. Qualquer campo pode ser utilizado como forma de busca. Adicionalmente, a busca deve ser feita considerando um ou mais campos. Por exemplo, é possível realizar a busca considerando somente o campo *codEstacao* ou considerando os campos *nomeEstacao* e *nomeLinha*. Em situações nas quais a busca for feita considerando o campo *codEstacao*, deve ser utilizado o arquivo de índice **indexaEstacao** para se fazer a busca. Tem-se, nesse caso, uma busca indexada. Para os demais casos, deve ser feita uma busca sequencial. Esta funcionalidade pode retornar 0 registros (quando nenhum satisfaz ao critério de busca), 1 registro (quando apenas um satisfaz ao critério de busca), ou vários registros. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas ("). Para a manipulação de *strings* com aspas duplas, pode-se usar a função *scan_quote_string* disponibilizada na página do projeto da disciplina. Para a busca por campos nulos, deve-se especificar o valor NULO. Registros marcados como logicamente removidos não devem ser exibidos. O arquivo de dados de entrada deve ser percorrido apropriadamente.

Sintaxe do comando para a funcionalidade [6]:

```
6 arquivoEntrada.bin arquivoIndicePrimario.bin n
m1 nomeCampo1 valorCampo1 ... nomeCampom1 valorCampom1
m2 nomeCampo1 valorCampo1 ... nomeCampom2 valorCampom2
...
mn nomeCampo1 valorCampo1 ... nomeCampomn valorCampomn
```

onde:

- arquivoEntrada.bin é o arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório.
- arquivoIndicePrimario.bin é um arquivo de índice primário no formato binário, o qual é gerado de acordo com as especificações da **Descrição do Arquivo de Índice Primário** definidas neste trabalho prático.
- n é a quantidade de vezes que a busca deve acontecer.
- m é a quantidade de vezes que o par nome do Campo e valor do Campo pode repetir em uma busca. Deve ser deixado um espaço em branco entre nomeCampo e valorCampo. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas (").
- Buscas por campos nulos devem ser especificadas usando o valor NULO.

Saída caso o programa seja executado com sucesso:

Cada registro deve ser mostrado em uma única linha e os seus campos devem ser mostrados de forma sequencial separado por um espaço em branco. Campos de tamanho fixo que tiverem o valor nulo devem ser exibidos da seguinte forma: ao invés de exibir o valor -1, escreva NULO. Campos de tamanho variável que tiverem o valor nulo devem ser exibidos da seguinte forma: NULO. A ordem de exibição dos campos dos registros deve ser codEstacao, nomeEstacao, codLinha, nomeLinha, codProxEstacao, distProxEstacao, codLinhaIntegra, codEstIntegra. Ver exemplo ilustrado no **exemplo de execução**.

Mensagem de saída caso não seja encontrado o registro que contém o valor do campo ou o campo pertence a um registro que esteja removido:

Registro inexistente.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab
6 estacao.bin indexaEstacao.bin 1
1 nomeEstacao "Luz"
```

9 Luz 1 Azul 10 NULO 4 55
55 Luz 4 Amarela 56 1257 1 9
111 Luz 7 Rubi 112 NULO NULO NULO
...

Na linguagem SQL, o comando DELETE é usado para remover dados em uma tabela. Para tanto, devem ser especificados quais dados (ou seja, registros) devem ser removidos, de acordo com algum critério.

DELETE FROM tabela (ou seja, arquivo que contém os campos)

WHERE critério de seleção (ou seja, critério de busca)

A funcionalidade [7] representa um exemplo de implementação do comando DELETE.

[7] Permita a remoção lógica de registros de um arquivo de dados de entrada, baseado na *abordagem dinâmica* de reaproveitamento de espaços de registros logicamente removidos. A implementação dessa funcionalidade deve ser realizada usando o conceito de *pilha de registros logicamente removidos*, e deve seguir estritamente a matéria apresentada em sala de aula. Os registros a serem removidos devem ser aqueles que satisfaçam um critério de busca determinado pelo usuário, sendo que a busca deve ser realizada conforme as especificações das funcionalidades [3] e [6]. Note que qualquer campo pode ser utilizado como forma de remoção. Ao se remover um registro, os valores dos *bytes* referentes aos campos já armazenados no registro devem permanecer os mesmos, com exceção dos valores dos campos relacionados ao tratamento da lista encadeada. Adicionalmente, as chaves de busca referentes aos registros logicamente removidos devem ser removidos do arquivo de índice primário criado na funcionalidade [5]. A funcionalidade [7] deve ser executada n vezes seguidas. Em situações nas quais um determinado critério de busca não seja satisfeito, ou seja, caso a solicitação do usuário não retorne nenhum registro a ser removido, o programa deve continuar a executar as remoções até completar as n vezes seguidas. Antes de terminar a execução da funcionalidade, deve ser utilizada a função `binarioNaTela`, disponibilizada na página do projeto da disciplina, para mostrar a saída do arquivo binário de dados.

Entrada do programa para a funcionalidade [7]:

```
7 arquivoEntrada.bin arquivoIndicePrimario.bin n
m1 nomeCampo1 valorCampo1 ... nomeCampom1 valorCampom1
m2 nomeCampo1 valorCampo1 ... nomeCampom2 valorCampom2
...
mn nomeCampo1 valorCampo1 ... nomeCampomn valorCampomn
```

onde:

- arquivoEntrada.bin é o arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório. As remoções a serem realizadas nessa funcionalidade devem ser feitas nesse arquivo.
- arquivoIndicePrimario.bin é um arquivo de índice primário no formato binário, o qual é gerado de acordo com as especificações da **Descrição do Arquivo de Índice Primário** definidas neste trabalho prático.
- n é o número de remoções a serem realizadas.
- m é a quantidade de vezes que o par nome do Campo e valor do Campo pode repetir na busca pelos registros a serem removidos. Deve ser deixado um espaço em branco entre o nome do campo e o valor do campo. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas (").

Saída caso o programa seja executado com sucesso:

Listar o arquivo de dados e o arquivo de índice no formato binário usando a função fornecida binarioNaTela.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab
7 estacao.bin indexaEstacao.bin 2
1 nomeEstacao "Luz"
2 nomeLinha "Verde" CodProxEst 27
```

usar a função binarioNaTela antes de terminar a execução da funcionalidade, para mostrar a saída dos arquivos estacao.bin e indexaEstacao.bin, os quais foram atualizados frente às remoções.

Na linguagem SQL, o comando INSERT INTO é usado para inserir dados em uma tabela. Para tanto, devem ser especificados os valores a serem armazenados em cada coluna da tabela, de acordo com o tipo de dado definido. A funcionalidade [8] representa exemplo de implementação do comando INSERT INTO.

[8] Permita a inserção de novos registros em um arquivo de dados de entrada, baseado na *abordagem dinâmica* de reaproveitamento de espaços de registros logicamente removidos. A implementação dessa funcionalidade deve ser realizada usando o conceito de *pilha de registros logicamente removidos*, e deve seguir estritamente a matéria apresentada em sala de aula. O lixo que permanece no registro logicamente removido e que não é reutilizado deve ser identificado pelo caractere '\$'. Adicionalmente, as chaves de busca referentes aos novos registros devem ser inseridas no arquivo de índice primário criado na funcionalidade [5]. Na entrada desta funcionalidade, os dados são referentes aos seguintes campos, na seguinte ordem: codEstacao, nomeEstacao, codLinha, nomeLinha, codProxEstacao, distProxEstacao, codLinhaIntegra, codEstIntegra. Campos com valores nulos, na entrada da funcionalidade, devem ser identificados com NULO. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas ("). Para a manipulação de *strings* com aspas duplas, pode-se usar a função `scan_quote_string` disponibilizada na página do projeto da disciplina. A funcionalidade [8] deve ser executada *n* vezes seguidas. Antes de terminar a execução da funcionalidade, deve ser utilizada a função `binarioNaTela`, disponibilizada na página do projeto da disciplina, para mostrar a saída do arquivo binário de dados.

Entrada do programa para a funcionalidade [8]:

```
8 arquivoEntrada.bin arquivoIndicePrimario.bin n
codEstacao1 nomeEstacao1 codLinha1 nomeLinha1 codProxEstacao1
distProxEstacao1 codLinhaIntegra1 codEstacaoIntegra1
codEstacao2 nomeEstacao2 codLinha2 nomeLinha2 codProxEstacao2
distProxEstacao2 codLinhaIntegra2 codEstacaoIntegra2
...
codEstacaon nomeEstacaon codLinhan nomeLinhan codProxEstacaon
distProxEstacaon codLinhaIntegran codEstacaoIntegran
```

onde:

- arquivoEntrada.bin é o arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório. As inserções a serem realizadas nessa funcionalidade devem ser feitas nesse arquivo.

- arquivoIndicePrimario.bin é um arquivo de índice primário no formato binário, o qual é gerado de acordo com as especificações da **Descrição do Arquivo de Índice Primário** definidas neste trabalho prático.

- n é o número de inserções a serem realizadas. Para cada inserção, deve ser informado os valores a serem inseridos no arquivo, considerando os seguintes campos, na seguinte ordem: codEstacao, nomeEstacao, codLinha, nomeLinha, codProxEstacao, distProxEstacao, codLinhaIntegra, codEstIntegra. Valores nulos devem ser identificados, na entrada da funcionalidade, por NULO. Cada uma das n inserções deve ser especificada em uma linha diferente. Deve ser deixado um espaço em branco entre os valores dos campos. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas (").

Saída caso o programa seja executado com sucesso:

Listar o arquivo de dados e o arquivo de índice no formato binário usando a função fornecida binarioNaTela.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab
8 estacao.bin indexaEstacao.bin 2
500 "Teste" 10 "Branca" NULO NULO NULO
501 "Nova Estacao" 10 "Branca" NULO NULO NULO NULO
usar a função binarioNaTela antes de terminar a execução da
funcionalidade, para mostrar as saídas dos arquivos estacao.bin e
indexaEstacao.bin, os quais foram atualizados frente às inserções.
```

Na linguagem SQL, o comando UPDATE é usado para atualizar dados em uma tabela. Para tanto, devem ser especificados quais valores de dados de quais campos devem ser atualizados, de acordo com algum critério de busca dos registros a serem atualizados.

UPDATE tabela (ou seja, arquivo que contém os dados)

SET quais colunas e quais valores (ou seja, quais campos e seus valores)

WHERE critério de seleção (ou seja, critério de busca)

A funcionalidade [9] representa um exemplo de implementação do comando UPDATE.

[9] Permita a atualização de registros de um arquivo de dados de entrada, baseado na *abordagem dinâmica* de reaproveitamento de espaços de registros logicamente removidos. A implementação dessa funcionalidade deve ser realizada usando o conceito de *pilha de registros logicamente removidos*, e deve seguir estritamente a matéria apresentada em sala de aula. Desde que os registros do arquivo de dados são de tamanho fixo, a atualização deve ser feita diretamente no registro existente que não esteja marcado como removido. O lixo que porventura permanecer no registro atualizado deve ser identificado pelo caractere '\$'. Adicionalmente, as chaves de busca referentes aos registros que foram atualizados devem ser atualizadas no arquivo de índice primário criado na funcionalidade [5]. Os registros a serem atualizados devem ser aqueles que satisfaçam um critério de busca determinado pelo usuário, sendo que a busca deve ser realizada conforme as especificações das funcionalidades [3] e [6]. Note que qualquer campo pode ser utilizado como forma de atualização. Adicionalmente, o campo utilizado como busca não precisa ser, necessariamente, o campo a ser atualizado. Por exemplo, pode-se buscar pelo campo *codEstacao*, e pode-se atualizar o campo *nomeLinha*. Campos a serem atualizados com valores nulos devem ser identificados, na entrada da funcionalidade, com NULO. A funcionalidade [9] deve ser executada n vezes seguidas. Em situações nas quais um determinado critério de busca não seja satisfeito, ou seja, caso a solicitação do usuário não retorne nenhum registro a ser atualizado, o programa deve continuar a executar as atualizações até completar as n vezes seguidas. Antes de terminar a execução da funcionalidade, deve ser utilizada a função *binarioNaTela*, disponibilizada na página do projeto da disciplina, para mostrar a saída do arquivo binário de dados.

Entrada do programa para a funcionalidade [9]:

```
9 arquivoEntrada.bin arquivoIndicePrimario.bin n
m1 nomeCampoB1 valorCampoB1 ... nomeCampoBm1 valorCampoBm1
p1 nomeCampoA1 valorCampoA1 ... nomeCampoAp1 valorCampoAp1
m2 nomeCampoB2 valorCampoB2 ... nomeCampoBm2 valorCampoBm2
p2 nomeCampoA1 valorCampoA1 ... nomeCampoAp2 valorCampoAp2
...
mn nomeCampoBn valorCampoBn ... nomeCampoBmn valorCampoBmn
pn nomeCampoAn valorCampoAn ... nomeCampoApn valorCampoApn
```

onde:

- arquivoEntrada.bin é o arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório. As atualizações a serem realizadas nessa funcionalidade devem ser feitas nesse arquivo.
- arquivoIndicePrimario.bin é um arquivo de índice primário no formato binário, o qual é gerado de acordo com as especificações da **Descrição do Arquivo de Índice Primário** definidas neste trabalho prático.
- n é o número de atualizações a serem realizadas.
- m é a quantidade de vezes que o par nomeCampoB (ou seja, nome do campo de busca) e valorCampoB (ou seja, valor do campo de busca) pode repetir na busca pelos registros a serem atualizados. Deve ser deixado um espaço em branco entre nomeCampoB e valorCampoB. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas (").
- p é a quantidade de vezes que o par nomeCampoA (nome do campo a ser atualizado) e valorCampoA (valor do campo a ser atualizado) pode repetir. Deve ser deixado um espaço em branco entre nomeCampoA e valorCampoA. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas (").

Saída caso o programa seja executado com sucesso:

Listar o arquivo de dados e o arquivo de índice no formato binário usando a função fornecida binarioNaTela.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab
9 estacao.bin indexaEstacao.bin 2
2 codEstacao 1 nomeEstacao "Tucuruvi"
3 codProxEstacao 15 codLinhaIntegrada 4 codEstacaoIntegrada 55
1 codEstacao 15
1 nomeEstacao "Exemplo"
usar a função binarioNaTela antes de terminar a execução da
funcionalidade, para mostrar as saídas dos arquivos estacao.bin e
indexaEstacao.bin, os quais foram atualizados frente às atualizações.
```

Restrições

As seguintes restrições têm que ser garantidas no desenvolvimento do trabalho.

[1] O arquivo de dados deve ser gravado em disco no **modo binário**. O modo texto não pode ser usado.

[2] Os dados do registro descrevem os nomes dos campos, os quais não podem ser alterados. Ademais, todos os campos devem estar presentes na implementação, e nenhum campo adicional pode ser incluído. O tamanho e a ordem de cada campo deve obrigatoriamente seguir a especificação.

[3] Deve haver a manipulação de valores nulos, conforme as instruções definidas.

[4] Não é necessário realizar o tratamento de truncamento de dados.

[5] Devem ser exibidos avisos ou mensagens de erro de acordo com a especificação de cada funcionalidade.

[6] Os dados devem ser obrigatoriamente escritos campo a campo. Ou seja, não é possível escrever os dados registro a registro. Essa restrição refere-se à entrada/saída, ou seja, à forma como os dados são escritos no arquivo.

[7] O(s) aluno(s) que desenvolveu(desenvolveram) o trabalho prático deve(m) constar como comentário no início do código (i.e. NUSP e nome do aluno). Para trabalhos desenvolvidos por mais do que um aluno, não será atribuída nota ao aluno cujos dados não constarem no código fonte.

[8] Todo código fonte deve ser documentado. A **documentação interna** inclui, dentre outros, a documentação de procedimentos, de funções, de variáveis, de partes do código

fonte que realizam tarefas específicas. Ou seja, o código fonte deve ser documentado tanto em nível de rotinas quanto em nível de variáveis e blocos funcionais.

[9] A implementação deve ser realizada usando a linguagem de programação C. As funções das bibliotecas `<stdio.h>` devem ser utilizadas para operações relacionadas à escrita e leitura dos arquivos. A implementação não pode ser feita em qualquer outra linguagem de programação. O programa executará no `[run.codes]`.

Material para Entregar

Arquivo compactado (a ser entregue no *run.codes*)

Deve ser preparado um arquivo `.zip` contendo:

- Código fonte do programa devidamente documentado.
- Makefile para a compilação do programa.

Vídeo (a ser entregue no *e-disciplinas*)

- Um vídeo gravado pelos integrantes do grupo, o qual deve ter, no máximo, 7 minutos de gravação. O vídeo deve explicar o trabalho desenvolvido. Ou seja, o grupo deve apresentar: cada funcionalidade e uma breve descrição de como a funcionalidade foi implementada. Todos os integrantes do grupo devem participar do vídeo, sendo que o tempo de apresentação dos integrantes deve ser balanceado. Ou seja, o tempo de participação de cada integrante deve ser aproximadamente o mesmo. O uso da webcam é obrigatório.

Instruções para fazer o arquivo makefile. No `[run.codes]` tem uma orientação para que, no makefile, a diretiva “all” contenha apenas o comando para compilar seu programa e, na diretiva “run”, apenas o comando para executá-lo. Adicionalmente, para utilizar a função `binarioNaTela`, é necessário usar a flag `-lmd`. Assim, a forma mais simples de se fazer o arquivo makefile é:

```
all:
```

```
gcc -o programaTrab *.c -lmd  
run:  
./programaTrab
```

Lembrando que *.c já engloba todos os arquivos .c presentes no seu zip. Adicionalmente, no arquivo Makefile é importante se ter um *tab* nos locais colocados acima, senão ele pode não funcionar.

Instruções de entrega.

O programa deve ser submetido via [run.codes]:

- página: <https://runcodes.icmc.usp.br/>
- Código de matrícula: **L6SW**

O vídeo gravado deve ser submetido por meio da página da disciplina no e-disciplinas, no qual o grupo vai informar o nome de cada integrante, o número do grupo e um link que contém o vídeo gravado. Ao submeter o link, verifique se o mesmo pode ser acessado. Vídeos cujos links não puderem ser acessados receberão nota zero. Vídeos corrompidos ou que não puderem ser corretamente acessados receberão nota zero.

Critério de Correção

Critério de avaliação do trabalho. Na correção do trabalho, serão ponderados os seguintes aspectos.

- Corretude da execução do programa.
- Atendimento às especificações do registro de cabeçalho e dos registros de dados.
- Atendimento às especificações da sintaxe dos comandos de cada funcionalidade e do formato de saída da execução de cada funcionalidade.
- Qualidade da documentação entregue. A documentação interna terá um peso considerável no trabalho.
- Estruturação da solução. O projeto deve apresentar uma organização adequada de arquivos e diretórios.

- Modularização. Trechos de código que aparecem repetidamente devem ser modularizados por meio de funções ou procedimentos, evitando duplicação de código.
- Uso de ferramentas de inteligência artificial. Não deve ser realizado para a implementação das funcionalidades especificadas neste trabalho e outras funções julgadas como necessárias para o embasamento do conteúdo ministrado. O uso dessas ferramentas acarretará desconto rigoroso na nota.
- Vídeo. Integrantes que não participarem da apresentação receberão nota 0 no trabalho correspondente.

Casos de teste no [run.codes]. Juntamente com a especificação do trabalho, serão disponibilizados 70% dos casos de teste no [run.codes], para que os alunos possam avaliar o programa sendo desenvolvido. Os 30% restantes dos casos de teste serão utilizados nas correções.

Restrições adicionais sobre o critério de correção.

- A não execução de um programa devido a erros de compilação implica que a nota final da parte do trabalho será igual a zero (0).
- O não atendimento às especificações do registro de cabeçalho e dos registros de dados implica que haverá uma diminuição expressiva na nota do trabalho.
- O não atendimento às especificações de sintaxe dos comandos de cada funcionalidade e do formato de saída da execução de cada funcionalidade implica que haverá uma diminuição expressiva na nota do trabalho.
- A ausência da documentação implica que haverá uma diminuição expressiva na nota do trabalho.
- A realização do trabalho prático com alunos de turmas diferentes implica que haverá uma diminuição expressiva na nota do trabalho.
- A inserção de palavras ofensivas nos arquivos e em qualquer outro material entregue implica que a nota final da parte do trabalho será igual a zero (0).
- Em caso de plágio, as notas dos trabalhos envolvidos serão zero (0).

Bom Trabalho!